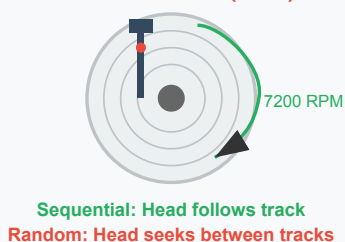


Database Storage Engines | System Design Primer

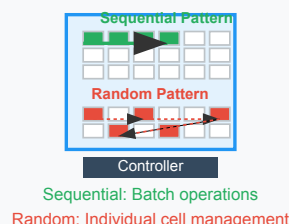
🌐 systemdesignschool.io/fundamentals/data-structures-behind-databases

HDD vs SSD Write Mechanics

Hard Disk Drive (HDD)



Solid State Drive (SSD)



Performance Characteristics

HDD Performance

Sequential Read: 150+ MB/s
Random Read: 1-2 MB/s
Seek Time: 8-15ms
Rotational Latency: 4-8ms
Why slow: Physical head movement

SSD Performance

Sequential Write: 500+ MB/s
Random Write: 200-300 MB/s
Access Time: 0.1ms
Why sequential wins:
• Batch erase/write operations

Data Structures Behind Databases

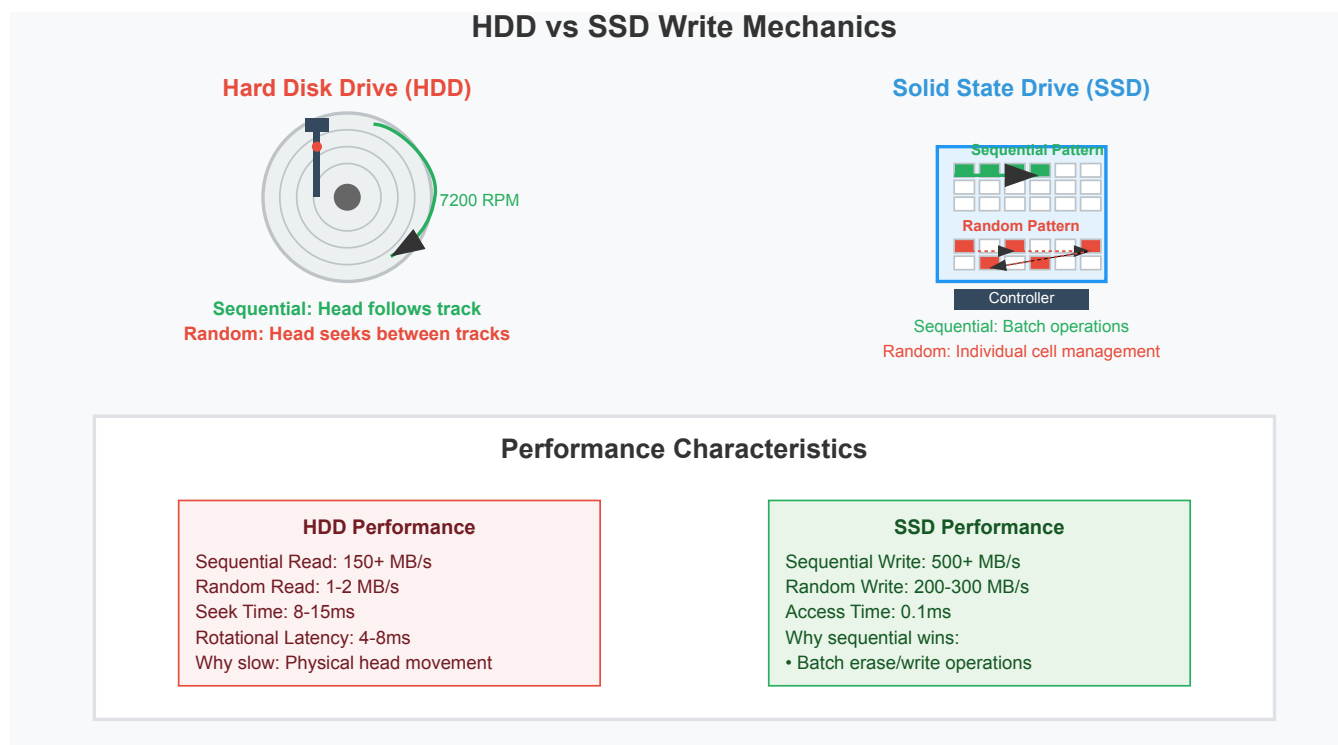
Note that this section is optional and provides an in-depth exploration of the data structures utilized in the construction of modern databases. It is not mandatory to possess this level of knowledge in order to solve system design questions. However, acquiring an understanding of these concepts will enhance your appreciation of various technologies and facilitate your comprehension of their documentations and learning new technologies.

Understanding the storage engine, the underlying technology that powers databases, may seem like an endeavor meant only for database developers. However, as a software engineer or a system designer, knowing these intricacies has significant benefits. It helps with database selection - understanding why PostgreSQL excels at reads while Cassandra handles massive writes enables informed choices. It also aids performance tuning by revealing how configuration changes affect underlying data structures.

Every database uses a storage engine that manages how data gets written to and read from disk. MySQL offers multiple storage engines (InnoDB, MyISAM, Memory) with different trade-offs. InnoDB uses B-trees for fast range queries, while RocksDB uses LSM trees for high write throughput. Sequential writes to disk are dramatically faster than random writes - often 10-100x on traditional drives. Even modern SSDs benefit from sequential access patterns through controller optimizations and wear leveling, though the gap is much smaller. This fundamental constraint drives database design decisions throughout the entire system.

Sequential Writes vs Random Writes

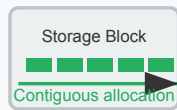
Physical storage hardware heavily favors sequential operations over random access. Traditional hard drives have physical read/write heads that must seek to different locations. Sequential writes keep the head moving in one direction, while random writes require expensive seek operations. SSDs eliminate mechanical movement but still benefit from sequential writes through better wear leveling and internal optimizations.



Kafka demonstrates this advantage in practice. Sequential writes to Kafka logs achieve 600 MB/sec on commodity hardware, while random database updates might only reach 60 MB/sec. This 10x difference drives architectural decisions across the entire system. Operating systems optimize for sequential access through larger buffer pools and predictive read-ahead. Database designers leverage these optimizations by structuring write patterns to align with filesystem behavior rather than fighting against it.

Sequential vs Random Write Performance

Sequential Writes



HDD: 150+ MB/s
SSD: 500+ MB/s

- No seek time
- Optimal for logs

Random Writes



HDD: 5-10 MB/s
SSD: 100-200 MB/s

- High seek overhead
- CPU/memory intensive

Real-World Impact

Kafka Logs
600 MB/s sequential writes
Append-only design

LSM Trees
Sequential writes + merging
Cassandra, RocksDB

B-Tree Updates
Random page modifications
Traditional RDBMS



Building a database from scratch

At its core, a database stores and retrieves data. Building a simple key-value store reveals the fundamental challenges that sophisticated data structures solve.

Simple File-Based Key-Value Store

```
from typing import Optional, Dict, Any
def __init__(self, db_dir: str = "simple_db"):
    self.db_dir = db_dir
    self.log_file = os.path.join(db_dir, "operations.log")
    self.index: Dict[str, int] = {}
    # Create database directory
    os.makedirs(db_dir, exist_ok=True)
    # Rebuild index from log file
    self._rebuild_index()
    def _rebuild_index(self):
        """Rebuild in-memory index from operation log"""
        if not os.path.exists(self.log_file):
            return
        with open(self.log_file, 'r') as f:
            for line in f:
                operation = json.loads(line.strip())
                if operation['type'] == 'set':
                    self.index[operation['key']] = operation['offset']
                elif operation['type'] == 'delete':
                    self.index.pop(operation['key'], None)
            offset = f.tell()
```

```
def set(self, key: str, value: Any) -> None:
    """Store a key-value pair"""
    'type': 'set',
```

Loading Python environment... This may take a few seconds on first load.

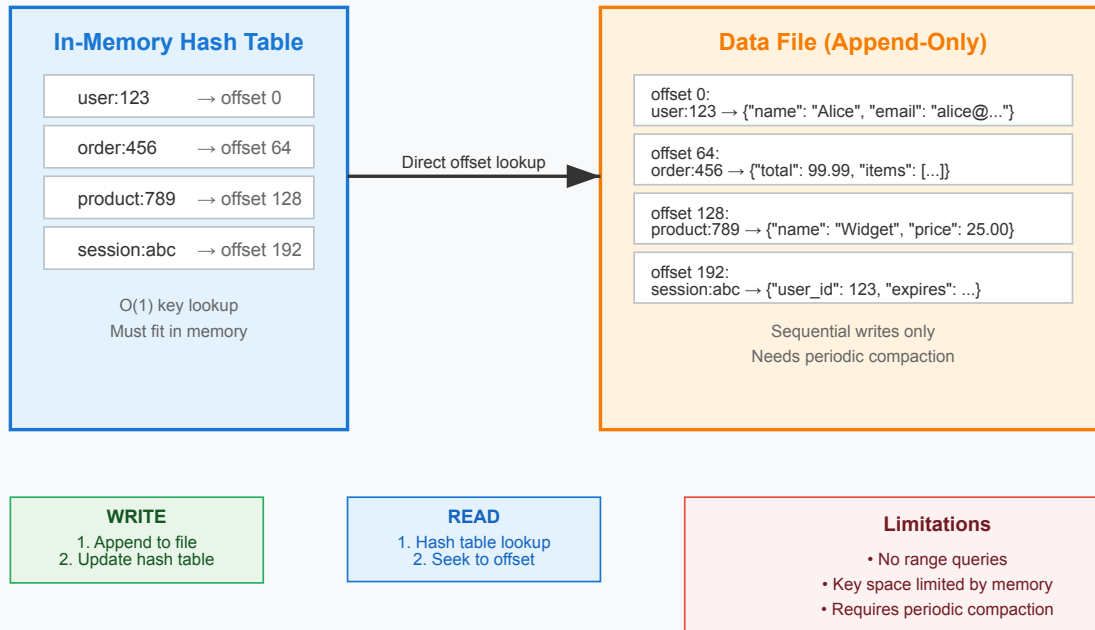
This simple implementation reveals fundamental database challenges. Sequential writes to the log file are fast, but the log grows indefinitely. The in-memory index enables quick lookups but must be rebuilt on restart. These constraints drive the need for more sophisticated data structures in production databases. Real databases solve these problems with techniques like log compaction, persistent indexes, and write-ahead logs. Understanding these basic trade-offs helps explain why different databases choose different approaches for their storage engines.

Hash Indexes

Hash indexes store key-value pairs using a hash table structure. Each key gets hashed to determine its storage location, enabling $O(1)$ average-case lookups. The database maintains an in-memory hash table that maps keys to file positions on disk. When storing data, the system appends records sequentially to data files and updates the hash table with the new record's location. Lookups hash the key, find the disk offset, and read the record directly.

Bitcask (used by Riak) implements this approach effectively. All writes append to log files while an in-memory hash table tracks key locations. This design provides extremely fast writes and reads for workloads with many random key lookups. However, the entire key space must fit in memory, limiting scalability. Hash indexes excel at point lookups but cannot answer range queries like "find all users with IDs between 1000 and 2000." The sequential write pattern keeps write performance high, but periodic log compaction is needed to reclaim space from deleted records.

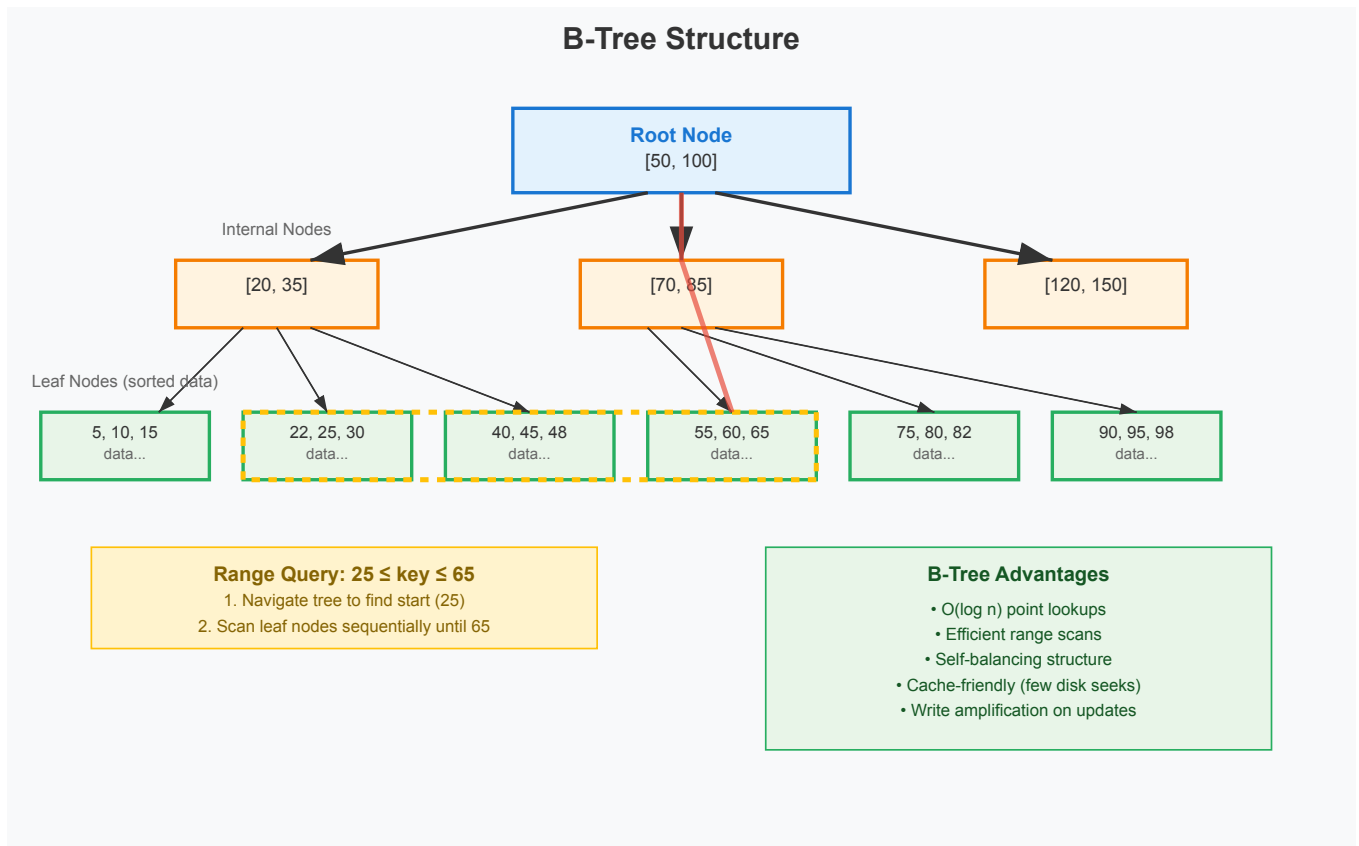
Hash Index Structure



B-Trees

B-trees organize data in sorted order across multiple disk pages, enabling both fast point lookups and efficient range scans. Most traditional relational databases use B-tree indexes as their primary storage structure. B-trees maintain balance automatically as data grows, ensuring $O(\log n)$ performance for reads, writes, and range queries. Each internal node contains multiple keys and pointers to child nodes, while leaf nodes store the actual data. This structure minimizes disk seeks by keeping frequently accessed nodes in memory.

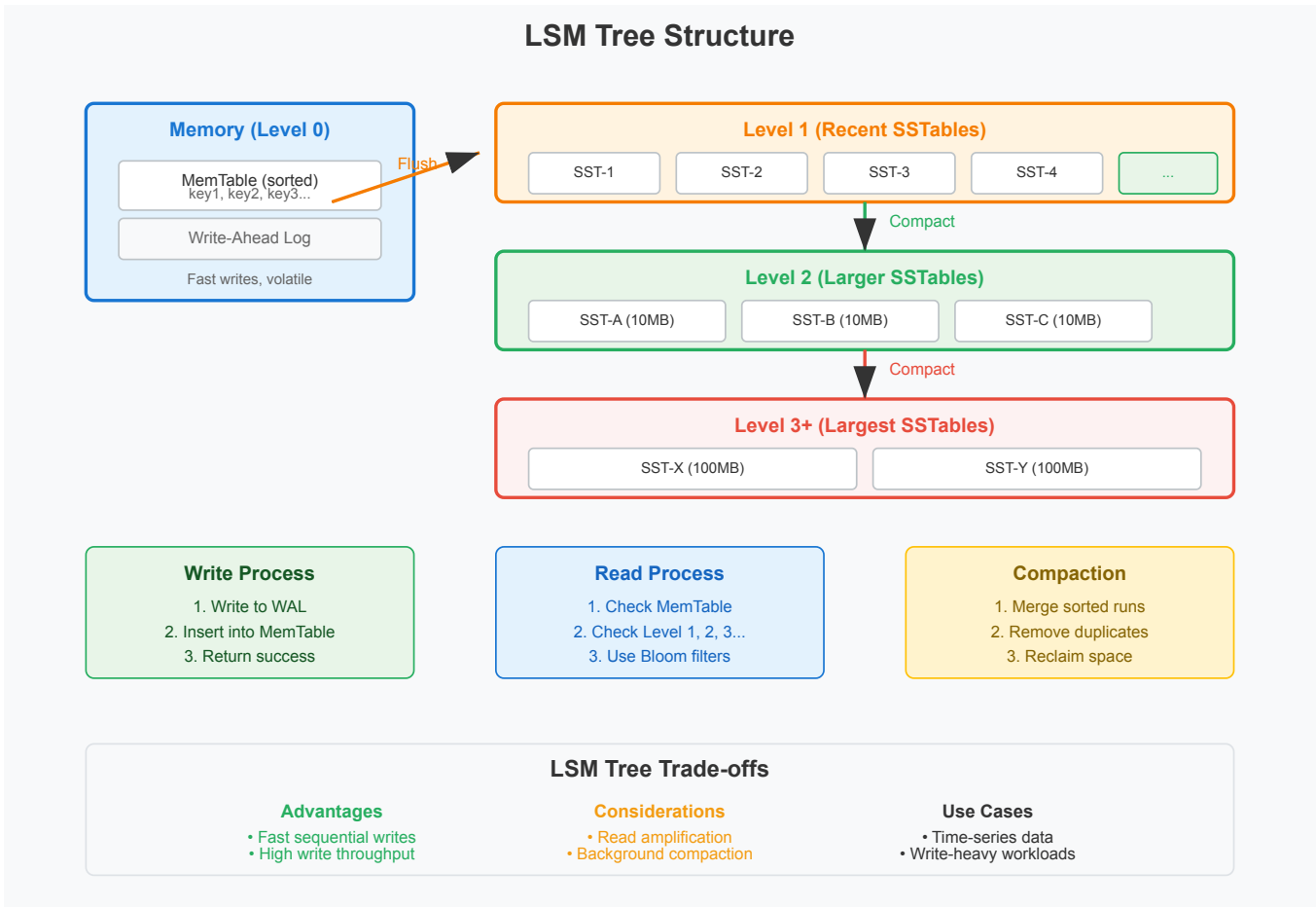
MySQL's InnoDB storage engine clusters table data using a B-tree where the primary key determines row placement. Secondary indexes point back to primary key values rather than row locations. This design optimizes for primary key lookups while supporting efficient secondary index operations. However, B-tree updates can trigger cascading changes up the tree structure. Inserting a single record might require updating multiple disk pages to maintain balance. This write amplification becomes problematic for write-heavy workloads, especially with traditional spinning disks.



LSM Trees

Log-Structured Merge trees optimize for write throughput by accepting temporary read penalties. Instead of updating data in place, LSM trees append writes to memory structures and periodically merge them to disk in sorted runs. LSM trees maintain data across multiple levels. New writes go to in-memory structures called memtables. When memtables fill up, they get flushed to disk as Level 0 sorted runs (SSTables). Background processes merge smaller runs into larger, sorted runs at higher levels. This structure keeps writes fast while eventually organizing data for efficient reads.

Apache Cassandra uses [LSM trees](#) to handle massive write workloads across distributed clusters. Writes first go to commit logs and memtables, providing immediate durability and fast acknowledgment. Background compaction processes merge SSTables (Sorted String Tables) to optimize read performance and reclaim space. LSM trees can require checking multiple sorted runs to find a record, creating read amplification. [Bloom filters](#) help skip files that definitely don't contain a key, while careful tuning of compaction strategies balances write performance against read latency.



Choosing the Right Structure

Different data structures optimize for different access patterns. Understanding these trade-offs helps select appropriate databases for specific workloads.

Structure	Write Performance	Read Performance	Range Queries	Use Cases
Hash Index	Excellent	Excellent	Not Supported	Cache stores, session data
B-Tree	Good	Excellent	Excellent	OLTP systems, traditional databases
LSM Tree	Excellent	Good	Good	Write-heavy analytics, time-series data

Modern databases often combine multiple storage structures. MongoDB uses B-trees for most indexes but supports hash indexes for specific use cases. RocksDB (an LSM tree implementation) powers various systems from MySQL storage engines to blockchain databases, proving the versatility of well-designed storage abstractions.